

Оглавление

Введение	2
1. Постановка задачи и анализ требований	4
1.1 Постановка задачи на разработку приложения	4
1.2 Анализ аналогов разрабатываемого приложения.....	6
1.3 Анализ функциональных требований	9
1.4 Анализ нефункциональных требований	15
1.5 Планирование разработки и оценка бюджета.....	16
2. Обучение по программам повышения квалификации	18
Заключение.....	19
Список использованных источников.....	20

Введение

Современный этап цифровизации сферы санаторно-курортного и гостиничного обслуживания характеризуется переходом от разрозненных инструментов (отдельных сайтов, электронных таблиц, локальных учётных программ) к единым цифровым платформам, обеспечивающим клиенту сквозной сценарий выбора, бронирования и оплаты размещения. Малые санаторно-гостиничные предприятия испытывают потребность в информационных системах, которые сочетают в себе персонализированный подбор варианта размещения, прозрачный учёт бронирований и контролируемый платёжный контур, и при этом допускают дальнейшее расширение без полной переработки программного продукта.

Актуальность темы выпускной квалификационной работы обусловлена необходимостью построения подобной платформы, ориентированной не на крупные сетевые гостиницы, а на сегмент малых санаторно-курортных учреждений. Такие предприятия, как правило, не располагают собственным цифровым контуром и вынуждены опираться на универсальные агрегаторы, не учитывающие специфику медицинского профиля, длительности оздоровительных программ и сезонных особенностей размещения. Это снижает эффективность подбора варианта размещения и затрудняет принятие клиентом обоснованного решения.

Объектом исследования являются процессы клиентского взаимодействия с санаторно-курортными учреждениями, включающие подбор санатория, оформление и сопровождение бронирования, а также расчётно-платёжные операции.

Предметом исследования выступают методы и средства проектирования и разработки информационной системы, обеспечивающей персонализированный подбор санаториев по совокупности критериев (город, медицинский профиль, удалённость от моря, ценовой диапазон, период проживания) и сквозное оформление бронирования с последующей оплатой.

Целью производственной (преддипломной) практики является постановка задачи на разработку программных средств персонализированного подбора и бронирования санаторно-курортных услуг, выполнение анализа предметной области, существующих аналогов и формирование функциональных и нефункциональных требований к разрабатываемой системе.

Для достижения поставленной цели в ходе практики решается ряд подзадач:

- провести анализ предметной области малого санаторно-курортного бизнеса и выделить ключевые бизнес-сценарии клиентского контура;
- выполнить постановку задачи на разработку программного продукта, определить состав основных функций и применяемых технологий;
- провести анализ существующих программных аналогов и обосновать целесообразность разработки собственного решения;
- сформулировать функциональные требования к разрабатываемой системе, описать варианты использования и сценарии работы пользователей;
- сформулировать нефункциональные требования, обеспечивающие безопасность, масштабируемость, надёжность и сопровождаемость системы;
- выполнить планирование разработки, рассчитать сроки и оценить бюджет проекта;
- оформить отчёт по практике в соответствии с требованиями, предъявляемыми к выпускной квалификационной работе бакалавра.

Успешное решение перечисленных подзадач позволяет сформировать обоснованную постановку задачи на разработку программных средств персонализированного подбора и бронирования санаторно-курортных услуг и определить технологическую основу для последующего проектирования и реализации системы в рамках выпускной квалификационной работы.

1. Постановка задачи и анализ требований

1.1 Постановка задачи на разработку приложения

В рамках выпускной квалификационной работы необходимо разработать информационную систему, предоставляющую клиенту инструмент персонализированного подбора и бронирования санаторно-курортных услуг. Система должна охватывать сквозной сценарий: от регистрации и поиска подходящего санатория до оформления договора и подтверждения оплаты. Архитектурно решение разделяется на серверную часть, реализованную на основе микросервисного подхода, и клиентскую часть, представленную одностраничным веб-приложением.

Разрабатываемая система должна реализовывать следующие функции:

- регистрация и аутентификация клиента, выдача токена доступа для последующих защищённых операций;
- персонализированный подбор санаториев по совокупности критериев: город, медицинский профиль, расстояние до моря, ценовой диапазон, период проживания, число гостей;
- предоставление детальной информации о санатории, включая состав услуг, медицинский профиль, расположение и фотогалерею;
- проверка фактической доступности санатория на заданные даты с учётом существующих бронирований и вместимости;
- создание, просмотр, изменение и отмена клиентских бронирований;
- оформление договора с клиентом сотрудником служебного контура (менеджером);
- автоматическое формирование счёта на оплату по факту оформления договора;
- идемпотентная обработка платежей с контролем соответствия суммы и автоматическим обновлением платёжного статуса договора;

- публикация доменных событий жизненного цикла бронирования и платежа для асинхронной согласованности данных между сервисами;
- разграничение доступа по ролям (клиент, менеджер, бухгалтер, администратор);
- предоставление REST-API с документацией в формате Swagger и служебных точек доступа мониторинга.

В качестве основного языка программирования серверной части необходимо использовать Go (Golang), так как он сочетает высокую производительность, встроенную поддержку конкурентности и эффективную работу с сетевыми протоколами, что соответствует требованиям микросервисной архитектуры. В качестве реляционной системы управления базами данных применяется PostgreSQL [1] -широко используемое промышленное решение, обеспечивающее необходимый уровень изоляции транзакций (Serializable) для критичных финансовых операций.

Для асинхронного взаимодействия между сервисами должен использоваться брокер сообщений NATS [2]. Он отличается лёгкостью развёртывания, низкими накладными расходами на доставку сообщений и поддержкой режима JetStream для гарантированной доставки доменных событий.

Клиентская часть системы должна быть реализована в виде одностраничного веб-приложения (Single-Page Application) на базе библиотеки React и языка программирования TypeScript. Такой подход обеспечивает строгую типизацию, повышает сопровождаемость кода и снижает количество ошибок на этапе разработки. В качестве сборщика проекта применяется Vite, обеспечивающий быструю горячую перезагрузку и оптимизированную production-сборку. Для управления состоянием серверных данных используется библиотека React Query, для глобального состояния клиента -Zustand, для маршрутизации -React Router, для стилизации -Tailwind CSS.

В качестве протокола межсервисного синхронного взаимодействия применяется REST поверх HTTP, обмен данными -в формате JSON [3]. Этот

формат является универсальным, поддерживается всеми современными языками программирования и удобен для чтения, отладки и документирования.

Все компоненты системы должны быть контейнеризированы с помощью Docker и оркестрированы средствами Docker Compose. Это обеспечивает воспроизводимость среды выполнения, упрощает локальную разработку и подготавливает основу для последующего развёртывания в кластерных средах.

В качестве среды разработки используется Visual Studio Code как универсальный кроссплатформенный редактор с обширной экосистемой расширений для языков Go и TypeScript, а также для работы с Docker и базами данных. Целевой операционной системой разработки выбрана Microsoft Windows 11, обеспечивающая совместимость со всеми перечисленными инструментами.

При выполнении всех вышеприведённых требований разрабатываемая информационная система будет обеспечивать реализацию всех заявленных возможностей и формировать целостный клиентский контур персонализированного подбора и бронирования санаторно-курортных услуг.

1.2 Анализ аналогов разрабатываемого приложения

В рамках работы над постановкой задачи проведён анализ существующих программных решений, обладающих функциональностью, частично или полностью пересекающейся с разрабатываемой системой. Основное назначение разрабатываемой системы заключается в персонализированном подборе санаториев по медицинским и потребительским критериям, оформлении бронирования и сопровождении платёжного контура для сегмента малых санаторно-курортных предприятий.

В качестве рассматриваемых аналогов выбраны следующие платформы:

- Booking.com -международный агрегатор средств размещения, поддерживающий поиск и бронирование гостиниц, апартаментов и иных объектов;

- Sanatory.ru -специализированный российский сервис подбора санаторно-курортных учреждений с возможностью фильтрации по медицинскому профилю;
- TUI Россия -туристический оператор с собственной платформой бронирования туров и санаторно-курортного отдыха;
- Tripadvisor -международная платформа отзывов и бронирования средств размещения с агрегированной оценкой пользователей.

Каждая из рассмотренных платформ обладает собственным набором сильных сторон, однако ни одна из них не обеспечивает одновременно специализированный подбор по медицинскому профилю, ориентацию на сегмент малых санаторно-курортных предприятий и возможность независимого развёртывания решения в инфраструктуре конкретного учреждения.

Для сравнения рассмотренных платформ выделен ряд критериев с установленной балльной оценкой:

- 0 - «полное отсутствие реализации»;
- 1 - «частичная реализация»;
- 2 - «полная реализация».

Далее, в таблице 1, приведено сравнение функциональных возможностей рассматриваемых платформ.

Таблица 1 – Сравнительная таблица приложений-аналогов

Параметр	Разрабатываемая система	Booking.com	Sanatory.ru	TUI	Trip-advisor
Подбор по медицинскому профилю	2	0	2	1	0
Фильтрация по расстоянию до моря	2	1	2	1	1
Фильтрация по ценовому диапазону	2	2	2	2	2
Сквозное оформление бронирования и оплаты	2	2	1	2	1
Ориентация на малые санаторно-курортные предприятия	2	0	1	0	0
Независимое развёртывание в инфраструктуре учреждения	2	0	0	0	0
Итого	12	5	8	6	4

Проведённое сравнение показывает, что существующие платформы реализуют только отдельные аспекты задачи.

Таким образом, Booking.com и Tripadvisor ориентированы на массовое размещение и не учитывают медицинскую специфику санаторно-курортных услуг. Sanatory.ru обеспечивает специализированный подбор, однако работает в формате агрегатора и не предоставляет малым учреждениям самостоятельного программного продукта. TUI Россия ориентирован на туристические пакеты и собственную сеть партнёров.

В отличие от рассмотренных аналогов, разрабатываемая система ориентирована именно на сегмент малых санаторно-курортных предприятий и предоставляет решение, ориентированное на развёртывание в инфраструктуре конкретного учреждения посредством контейнеризации (Docker Compose). Помимо специализированного подбора по медицинскому профилю и расстоянию до моря, разрабатываемая система предоставляет открытый REST API со Swagger-документацией, что позволяет интегрировать её со сторонними учётными системами учреждения.

В результате анализа аналогов установлено, что существующие решения не обеспечивают одновременной реализации всех необходимых функций в рамках одного программного продукта, ориентированного на малые санаторно-курортные предприятия. Это подтверждает актуальность разработки данного программного обеспечения.

Разрабатываемая система обладает более широкими функциональными возможностями за счёт сочетания персонализированного подбора по медицинскому профилю, сквозного оформления бронирования и оплаты, а также ориентации на самостоятельное развёртывание в инфраструктуре учреждения, что делает её универсальным инструментом цифровизации клиентского контура санаторно-курортных предприятий.

1.3 Анализ функциональных требований

В процессе анализа функциональных требований на основе поставленной задачи были выделены следующие сценарии работы пользователей системы:

- зарегистрироваться в системе;
- пройти аутентификацию и получить токен доступа;
- открыть каталог санаториев;
- задать параметры персонализированного подбора (город, медицинский профиль, расстояние до моря, ценовой диапазон, даты проживания, число гостей);
- получить список санаториев, соответствующих заданным критериям;
- просмотреть детальную информацию о выбранном санатории;
- проверить доступность санатория на заданный период;
- создать бронирование на выбранные даты;
- просмотреть список собственных бронирований;
- изменить параметры существующего бронирования;
- отменить ранее созданное бронирование;
- оформить договор с клиентом (сценарий служебного контура);
- зафиксировать оплату счёта по договору (сценарий служебного контура);
- получить документацию по REST API системы;
- получить сведения о работоспособности компонентов системы (для администратора).

Для демонстрации выделенных сценариев была составлена диаграмма вариантов использования [4] (рисунок 1).

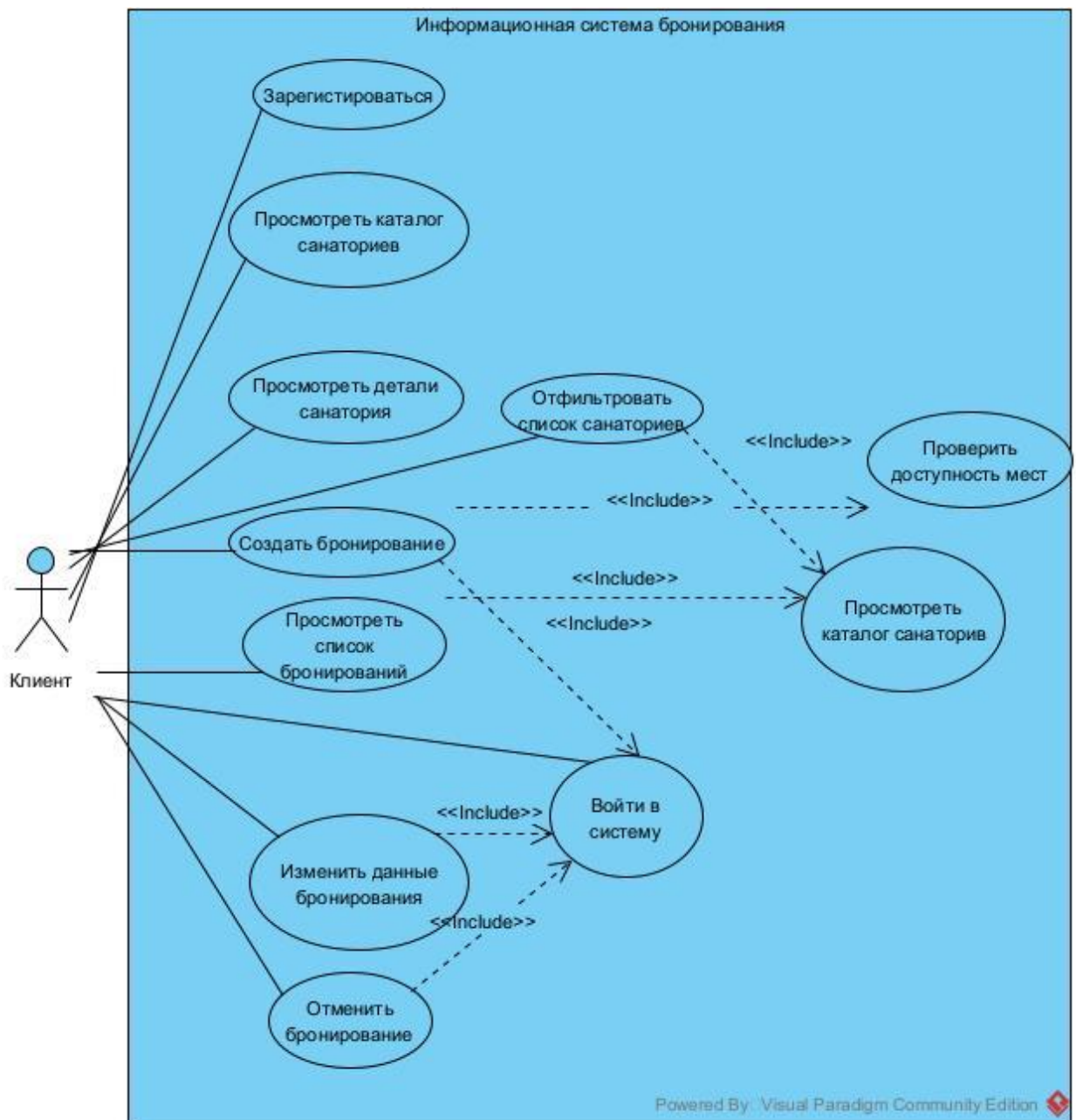


Рисунок 1 – Диаграмма вариантов использования

На рисунках 2-6 представлено описание некоторых наиболее важных сценариев вариантов использования (далее ВИ) разрабатываемого приложения.

Прецедент: создать бронирование
ID: 7
Краткое описание: клиент выбирает санаторий, указывает даты и создаёт бронирование. Включает проверку доступности мест, расчёт стоимости и оплату.
Действующие лица: клиент
Предусловие: клиент аутентифицирован; санаторий выбран в каталоге
Основной поток: 1. Клиент указывает дату заезда, дату выезда и количество гостей. 2. Система проверяет корректность дат и наличие свободных мест на указанный период. 3. Система рассчитывает стоимость бронирования исходя из цены санатория и длительности проживания. 4. Клиент подтверждает бронирование и переходит к оплате. 5. Система создаёт бронирование в статусе confirmed и публикует событие в брокер сообщений. 6. Система возвращает клиенту подтверждение с деталями бронирования.
Постусловие: бронирование создано и отображается в списке бронирований клиента.

Рисунок 2 – Описание ВИ «Создать бронирование»

Прецедент: изменить данные бронирования
ID: 3
Краткое описание: клиент пожелал изменить данные уже созданного бронирования: даты заезда/выезда или количество гостей
Действующие лица: клиент
Предусловие: клиент аутентифицирован; бронирование существует, принадлежит клиенту и находится в статусе confirmed
Основной поток: 1. Клиент открывает список своих бронирований 2. Клиент выбирает бронирование и переходит к его редактированию 3. Клиент изменяет параметры: дату заезда, дату выезда или количество гостей 4. Система проверяет корректность новых дат и наличие свободных мест на изменённый период с учётом существующих бронирований. 5. Система сохраняет обновлённые параметры бронирования 6. Система возвращает клиенту подтверждение с обновлёнными данными
Постусловие: Параметры бронирования обновлены и отражены в личном кабинете клиента.

Рисунок 3 – Описание ВИ «Изменить данные бронирования»

На рисунках 2 и 3 были описаны операции, связанные с бронированием. Далее, на рисунках 4 и 5 приводятся прецеденты, связанные с авторизацией клиента.

Прецедент: зарегистрироваться
ID: 1
Краткое описание: клиент создаёт учётную запись в системе, вводя адрес электронной почты, пароль и полное имя.
Действующие лица: клиент
Предусловие: клиент открыл страницу регистрации
Основной поток: 1. Клиент вводит адрес электронной почты, пароль и полное имя. 2. Система проверяет корректность формата email и наличие обязательных полей. 3. Система передаёт данные в сервис аутентификации, который сохраняет учётную запись (пароль хранится в виде bcrypt-хеша). 4. Система уведомляет клиента об успешной регистрации и предлагает перейти ко входу в систему.
Постусловие: учётная запись клиента создана; клиент может выполнить вход в систему.

Рисунок 4 – Описание ВИ «Зарегистрироваться»

Прецедент: войти в систему
ID: 2
Краткое описание: клиент вводит учётные данные и получает JWT-токен для доступа к защищённым функциям.
Действующие лица: клиент
Предусловие: учётная запись клиента создана
Основной поток: 1. Клиент вводит адрес электронной почты и пароль. 2. Система проверяет учётные данные через сервис аутентификации. 3. Сервис аутентификации проверяет пароль и генерирует JWT-токен с идентификатором пользователя и ролью. 4. Клиент получает токен доступа и переходит в личный кабинет.
Постусловие: клиент аутентифицирован; токен доступа сохранён и используется при последующих операциях.

Рисунок 5 – Описание ВИ «войти в систему»

На рисунках 4 и 5 были описаны наиболее важные прецеденты при авторизации клиента. Далее, логически следует, что клиент станет искать нужные ему санатории. Данный прецедент приведён на рисунке 6.

Прецедент: отфильтровать список санаториев
ID: 4
Краткое описание: клиент задаёт параметры фильтрации и получает список санаториев, соответствующих заданным критериям.
Действующие лица: клиент
Предусловие: клиент открыл страницу каталога санаториев
Основной поток: 1. Клиент задаёт один или несколько параметров фильтрации: город, медицинский профиль, расстояние до моря, ценовой диапазон, даты проживания, количество гостей. 2. Система отправляет запрос с параметрами фильтрации на сервер. 3. Сервер возвращает список санаториев, соответствующих критериям, с пагинацией. 4. Клиент просматривает результаты и при необходимости уточняет параметры подбора.
Постусловие: клиент получил список подходящих санаториев и может перейти к просмотру деталей.

Рисунок 6 – Описание ВИ «Отфильтровать список санаториев»

Спроектированные варианты использования будут реализованы в ходе разработки приложения.

1.4 Анализ нефункциональных требований

Разрабатываемая система должна соответствовать ряду нефункциональных требований, обеспечивающих её надёжную, безопасную и сопровождаемую работу:

- **Безопасность:** доступ к защищённым функциям должен быть ограничен. Аутентификация клиентов и сотрудников служебного контура должна выполняться на основе JWT-токенов [5], межсервисное взаимодействие -защитаться внутренним API-ключом, пароли пользователей -храниться в виде bcrypt-хеша.

- **Масштабируемость:** должна быть обеспечена возможность независимого расширения и развёртывания отдельных подсистем. Это требование удовлетворяется выделением независимых микросервисов (аутентификация, бронирования и каталог, оплата) и их размещением в Docker-контейнерах.

- **Надёжность:** отказ одной подсистемы не должен приводить к отказу системы в целом. Критичная бизнес-логика (оплата счёта) должна быть защищена от гонок данных с применением транзакций PostgreSQL уровня изоляции Serializable и идемпотентных ключей для платежей.

- **Наблюдаемость:** каждый микросервис должен предоставлять точки доступа мониторинга (/health, /health/ready и /metrics в формате Prometheus), что позволяет интегрировать систему со сторонними средствами мониторинга и алертинга.

- **Сопровождаемость:** сервисы должны строиться по единой многослойной архитектуре (transport → service → repository → domain) с формализованными контрактами REST API и Swagger-документацией.

- **Производительность:** время отклика системы при поиске санаториев и просмотре каталога не должно превышать одной секунды при нагрузке до ста одновременных пользователей.

- **Совместимость:** клиентская часть должна корректно работать в актуальных версиях браузеров (Google Chrome, Mozilla Firefox, Microsoft Edge) на персональных компьютерах и мобильных устройствах.

- **Удобство использования:** клиентский интерфейс должен быть интуитивно понятным, поддерживать адаптивную вёрстку и обеспечивать минимальное количество шагов до оформления бронирования.

Реализация всех перечисленных нефункциональных требований обеспечит стабильную, надёжную и сопровождаемую работу разрабатываемой системы.

1.5 Планирование разработки и оценка бюджета

Для отражения основных этапов разработки информационной системы персонализированного подбора и бронирования санаторно-курортных услуг в соответствии с заданными требованиями и технологиями составлена диаграмма Ганта плана разработки в среде Microsoft Office Project (рисунок 6).

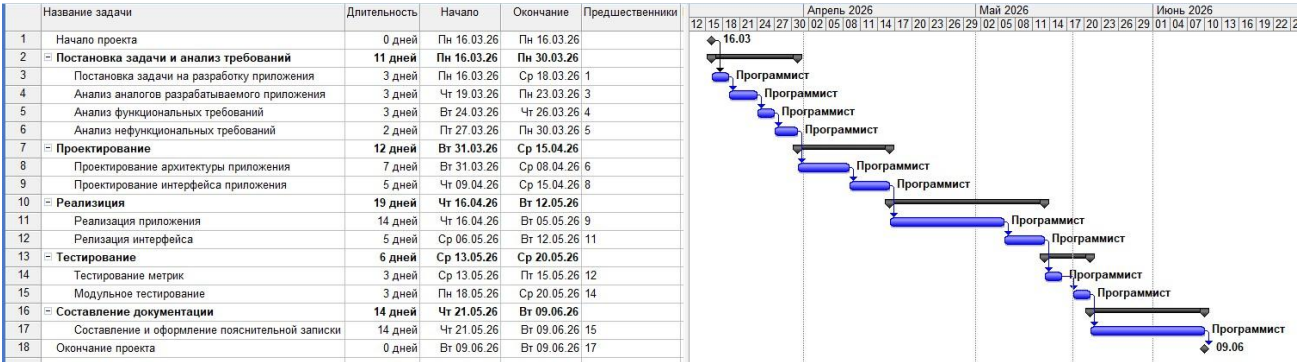


Рисунок 7 – Диаграмма Ганта плана разработки

На диаграмме можно увидеть все этапы разработки приложения, начиная от постановки задачи и анализа требований, и заканчивая тестированием и составлением пояснительной записки.

В разработке приложения участвует только один человек – программист, со ставкой 200 рублей в час.

Итоговая статистика разработки проекта представлена на рисунке 9.

	Длительность	Трудозатраты	Затраты
Текущие	62д	496ч	р.99 200,00
Базовые	0д?	0ч	р.0,00
Фактические	0д	0ч	р.0,00
Оставшиеся	62д	496ч	р.99 200,00

Рисунок 9 – Статистика проекта

Подводя итог, можно видеть, что разработка андроид-приложения сбора данных о радиообстановке ближайшего окружения займет 62 рабочих дня, что соответствует 496 часам. Бюджет разработки составит почти 100 тысяч рублей.

2. Обучение по программам повышения квалификации

В соответствии с приказом 98/д от 06.04.2026 «О проведении обучения по программам повышения квалификации студентов выпускных курсов университета» в период прохождения производственной (преддипломной) практики пройдено обучение по программам повышения квалификации «Профессиональная этика и этикет» и «Деловой русский язык». Результаты прохождения итогового тестирования приведены на рисунках 10 и 11.

The screenshot shows the Moodle interface for a user named Артём Эдуардович Пилогин. The page title is 'Профессиональная этика и этикет'. The breadcrumb trail is: Кабинет moodle / Мои курсы / ДПО / МРЦПКидО / Программы повышения квалификации для студентов 2026 / Профессиональная этика и этикет / Итоговая аттестация / Тест. The page content shows 'Тест' with a 'Метод оценивания: Высшая оценка'. Below this, a table titled 'Результаты ваших предыдущих попыток' shows one attempt: Attempt 1, Status: Completed (Отправлено Среда, 20 мая 2026, 11:32), Score: 31,00 / 33,00, View: Просмотр, Feedback: пройден. Below the table, it says 'Высшая оценка: 31,00 / 33,00. Итоговый отзыв пройден' and there is a button 'Пройти тест заново'.

Попытка	Состояние	Оценка / 33,00	Просмотр	Отзыв
1	Завершены Отправлено Среда, 20 мая 2026, 11:32	31,00	Просмотр	пройден

Высшая оценка: 31,00 / 33,00.
Итоговый отзыв
пройден

[Пройти тест заново](#)

Рисунок 10 – Результаты прохождения итоговой аттестации по программе

«Профессиональная этика и этикет»

The screenshot shows the Moodle interface for the same user. The page title is 'Деловой русский язык'. The breadcrumb trail is: Кабинет moodle / Мои курсы / ДПО / МРЦПКидО / Программы повышения квалификации для студентов 2026 / Деловой русский язык / Итоговая аттестация / Итоговое тестирование. The page content shows 'Итоговое тестирование' with a 'Метод оценивания: Высшая оценка'. Below this, a table titled 'Результаты ваших предыдущих попыток' shows one attempt: Attempt 1, Status: Completed (Отправлено Среда, 20 мая 2026, 11:39), Score: 20,00 / 20,00, View: Просмотр, Feedback: пройден. Below the table, it says 'Высшая оценка: 20,00 / 20,00. Итоговый отзыв пройден' and there is a button 'Пройти тест заново'.

Попытка	Состояние	Оценка / 20,00	Просмотр	Отзыв
1	Завершены Отправлено Среда, 20 мая 2026, 11:39	20,00	Просмотр	пройден

Высшая оценка: 20,00 / 20,00.
Итоговый отзыв
пройден

[Пройти тест заново](#)

Рисунок 11 - Результаты прохождения итоговой аттестации по программе

«Деловой русский язык»

Заключение

В ходе прохождения производственной (преддипломной) практики выполнена работа, связанная с постановкой задачи на разработку программных средств персонализированного подбора и бронирования санаторно-курортных услуг и анализом соответствующих требований.

Изначально были сформулированы задачи, которые необходимо выполнить в ходе разработки программного продукта. Далее были рассмотрены программные аналоги будущего приложения. В конце был проведен анализ функциональных и нефункциональных требований.

Также в рамках прохождения практики были пройдены программы повышения квалификации для студентов 2026 по направлениям «Деловой русский язык» и «Профессиональная этика и этикет». По итогам прохождения программ была успешно пройдена итоговая аттестация.

Список использованных источников

1. PostgreSQL: The World's Most Advanced Open Source Relational Database - URL: <https://www.postgresql.org/docs/>
2. NATS - Connective Technology for Adaptive Edge & Distributed Systems - URL: <https://docs.nats.io/>
3. Introducing JSON : JSON - URL: <https://www.json.org/json-en.html>
4. Арлоу, Д. UML 2 и Унифицированный процесс. Практический объектно-ориентированный анализ и проектирование. / Д. Арлоу, И. Нейштадт. - СПб: Символ-Плюс, 2007. - 624 с.
5. Jones, M. JSON Web Token (JWT) : RFC 7519 - URL: <https://datatracker.ietf.org/doc/html/rfc7519>